Agent-Based Research Monitoring for a RAG Benchmarking Framework using Hermes Agent

I. Research Context and Problem

My research centres on RAG-NIMBF [1], an open-source, non-invasive benchmarking framework that sweeps the RAG configuration space (chunking, embedding, retrieval, reranking, prompting, generator) and scores each configuration with LLM-based quality metrics in the style of RAGAS [2], retrieval metrics (nDCG@k) and runtime, tracked in MLflow. Its value depends on covering the current state of the art, yet new rerankers, chunking schemes, embedders and evaluation protocols [3] appear weekly; manual screening is slow and unsystematic.

II. Open-Source Tool: Hermes Agent

Hermes Agent (Nous Research, MIT license [4]) is a self-hostable LLM agent runtime. Four of its features make it suitable for research automation: (i) a *cron* scheduler whose jobs can run deterministic pre-scripts and skip the LLM entirely when nothing changed (`wakeAgent`=false), keeping cost near zero; (ii) versionable *skills* that hold the research methodology separately from job prompts; (iii) persistent memory and a Kanban board for multi-stage pipelines; and (iv) delivery to messaging channels and a Git-versioned Obsidian vault. It already runs on my IONOS VPS with local or API models.

III. Proposed Setup

The pipeline (Fig. 1) consists of four scheduled jobs: **(1) Collectors** query the arXiv API (cs.IR, cs.CL), Semantic Scholar (new citers of RAGAS, ARES, RAG-NIMBF) and GitHub releases of the RAG stack, deduplicated by ID. Besides improvements on existing axes, they explicitly target *new RAG techniques* (e.g. graph-based, agentic or self-reflective RAG) that may require new axes. **(2) Relevance evaluation** scores each item with a written rubric (axis fit or novelty, evidence quality, reproducibility and license, feasibility on consumer GPUs, integration effort); items below $7/10$ are only logged. Claims are quoted from the source and arXiv IDs verified to prevent hallucinated papers. **(3) Summaries**: for each relevant paper, the agent stores the PDF (open-access sources only) and a `summary.md` (core idea, results, rubric score with justification, affected axis) in a GitHub repository under `papers/<id>/`. **(4) Fit check and proposals**: once a week the agent selects the summarised papers that can actually be expressed in RAG-NIMBF and only for these writes a `config.yaml` with a falsifiable hypothesis (e.g. "a cross-encoder reranker raises nDCG@5 by $\geq$3 pp"), committed after my approval. This two-stage filter avoids an information overload.

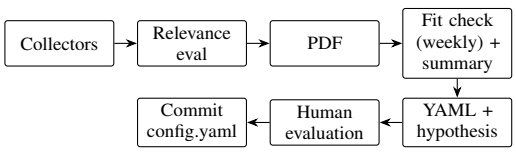


Fig. 1. RAG Radar pipeline. Each stage filters; the agent triggers no benchmark runs.

IV. Benefit for My Research

- **Coverage:** systematic, documented monitoring instead of ad-hoc reading; the repository's Git history provides an audit trail of what was screened and why.
- **Focus:** short summaries and YAML only for fitting papers, no overload.
- **Publications:** a monthly gap report feeds Related and Future Work of the next RAG-NIMBF paper (e.g. HyDE, reranking).

References

[1] N. Hart and N. Weeger, "RAG-NIMBF: A non-invasive, modular benchmarking framework for automated RAG configuration analysis," 2026. [Online]. Available: https://github.com/aiengineeringblueprints/RAG-NIMBF

[2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proc. EACL: System Demonstrations*, 2024, pp. 150–158.

[3] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," in *Proc. NAACL*, 2024.

[4] Nous Research, "Hermes Agent documentation," 2026. [Online]. Available: https://hermes-agent.nousresearch.com/docs